

Indexing All (r, s) -Nucleus Decompositions

Anonymous Author(s)

Abstract

Dense subgraphs carry much of the meaning of a graph, and decomposing a graph into hierarchies of dense subgraphs is a fundamental task in graph mining. The (r, s) -nucleus decomposition is the clique-level member of this family, scoring every r -clique by its surrounding s -cliques. It subsumes both the k -core and the k -truss. The parameter pair (r, s) acts as a resolution, and analysts need many cells of the parameter plane rather than one. Existing algorithms compute one cell at a time and materialize every r -clique of that cell. A plane is therefore out of reach: a seven-million-edge web graph already contains more than 10^{12} 5-cliques. We observe that vertices belonging to exactly the same maximal cliques are interchangeable, and that the whole plane respects this symmetry. Based on this observation, we propose NSI, an index built on the clique quotient of the graph. Cliques collapse into patterns, support counts collapse into polynomial coefficients, and two transfer theorems reduce the whole plane to a single peeled cell. Wherever the closed form is certified, the index stores nothing and reconstructs the answer at query time. On real graphs the index is up to seven orders of magnitude smaller than the explicit table it replaces, and it answers point queries in tens of nanoseconds. Its builder outperforms the state-of-the-art single-cell algorithm by up to $742\times$ in time and $226\times$ in memory. A one-number test, computable before any index is built, tells in advance whether a graph is suitable.

1 Introduction

A graph represents entities as vertices and their pairwise relationships as edges. Groups of vertices that are densely connected to one another often carry the meaning of the whole network. For example, in collaboration networks, dense groups correspond to research teams working closely together [5]; in web graphs, they mark clusters of pages on one topic [4]; in biological networks, they capture protein complexes [10]; in financial and social platforms, they expose coordinated fraud rings [1, 6].

Decomposing a graph into a hierarchy of nested dense subgraphs is a fundamental task in graph mining. Representative applications include community detection [5, 11], densest-subgraph discovery [3], influence analysis [7, 8], and graph visualization [9]. In the literature, the task has been studied at three levels of granularity. The k -core peels vertices by degree [2, 15]; the k -truss peels edges by triangle count [16, 18]; the (r, s) -nucleus decomposition peels r -cliques by s -clique count [13]. The nucleus model subsumes the other two: the $(1, 2)$ -nucleus is exactly the k -core, and the $(2, 3)$ -nucleus is exactly the k -truss [13]. The parameter pair (r, s) therefore acts as a resolution, and every cell of the parameter plane reveals the graph’s dense structure at a different granularity [13, 14]. The state-of-the-art algorithm, CND [17], computes the decomposition of one cell at a time. In this paper, we aim to answer every cell of a bounded parameter plane from one index.

For integers $r < s$, consider subgraphs in which every r -clique is contained in at least k s -cliques of the same subgraph. The nucleus value of an r -clique is the largest k for which such a subgraph

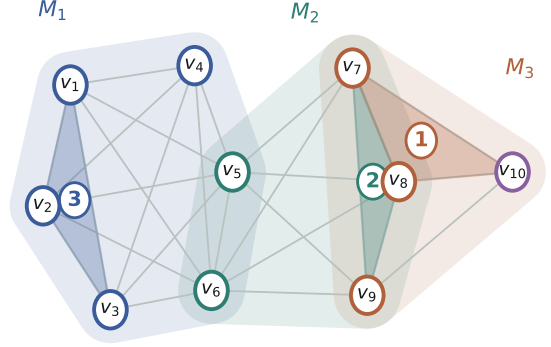


Figure 1: An example graph G and three of its $(3, 4)$ -nucleus values.

contains it. Computing the cell (r, s) means computing this value for every r -clique of the graph.

Example 1.1. Consider the graph G in Figure 1, whose ten vertices form three maximal cliques: M_1 with six vertices, M_2 with five, and M_3 with four. At the cell $(3, 4)$, the triangle $\{v_1, v_2, v_3\}$ has nucleus value 3: it lies in the subgraph induced by M_1 , in which every triangle is contained in at least three 4-cliques. The triangle $\{v_7, v_8, v_9\}$ has value 2, and the triangle $\{v_7, v_8, v_{10}\}$ has value 1. Other cells give other resolutions: at $(3, 5)$ the value of $\{v_1, v_2, v_3\}$ is still 3, while at $(4, 5)$ the 4-clique $\{v_1, v_2, v_3, v_4\}$ has value 2.

Applications. Many uses of the decomposition need several cells of the plane rather than one. *Multi-resolution exploration.* An analyst inspecting a dense region rarely knows the right resolution in advance. Low cells surface broad communities, while high cells isolate small, strongly coordinated groups [6, 13]. Exploration therefore issues a stream of queries across cells, and each query must return in interactive time. *Parameter selection.* Which cell best separates signal from noise differs across graphs and tasks [14]. Practitioners choose by probing: compute several cells, compare the resulting hierarchies, and keep the most informative one. Today every probe costs a full decomposition run. *Point queries at query time.* Downstream systems ask point questions: given one concrete group of vertices, how dense is its surrounding structure at each resolution [12]? Answering from the raw graph requires a fresh decomposition per question.

Existing Methods and Key Limitation. The state-of-the-art algorithm CND [17] enumerates every r -clique of the requested cell, counts its supporting s -cliques, and repeatedly removes the clique of minimum count. Careful counting structures let it obtain these counts without listing individual s -cliques, which made single-cell decomposition practical on graphs with millions of edges [17]. However, all existing algorithms share a fundamental limitation: they materialize *every r -clique of every cell they compute*. The number of r -cliques grows combinatorially with r : web-it-2004, a web graph

with 7.2 million edges, contains more than 1.5×10^{12} 5-cliques. Even a single shallow cell is already prohibitive: on web-it-2004, CND needs 5,581 seconds and 101 GB of memory for the single cell (3, 4). Within a 1,800-second, 300 GB budget, CND finishes none of the nine web-it-2004 cells of our evaluation grid (Section 8).

This leads to the central research question of this paper: **Can one index answer the nucleus value of every r -clique, at every cell of a bounded plane, without ever materializing the plane?**

Challenges. Answering the plane from one structure faces three obstacles. Astronomically many objects. For one small cell, per-clique computation is affordable. However, across a plane the objects multiply twice: the number of r -cliques explodes with r , and every cell owns its own copy of the work. Even the ten-vertex graph of Figure 1 carries 61 cliques of size three to five; on web-it-2004 the count exceeds 10^{12} . Therefore, any approach that touches each r -clique of each cell even once is ruled out from the start. Counting without enumeration.

Peeling is driven by counts: each removal must decrement the count of every r -clique that shared a supporting s -clique with the removed one. However, s -cliques outnumber r -cliques, and at high s they cannot even be listed once. Therefore, both the initial counts and every later decrement must be obtained without enumerating a single s -clique. A plane of cells. A bounded plane contains dozens of cells, and each cell is a full decomposition in its own right. However, running even the best single-cell algorithm once per cell multiplies its cost by the number of cells. Worse, the deep cells are exactly the ones no existing algorithm reaches. Therefore, the computation done in one cell must be made to pay for all the others.

Our Idea. Our solution is built on one structural observation: vertices that belong to exactly the same maximal cliques are interchangeable, and the whole plane respects this symmetry. We call the resulting grouping the *clique quotient* of the graph. Peel patterns, not cliques.

Under the quotient, an r -clique is described by how many vertices it takes from each group, not by which vertices it takes. The 33 triangles of Figure 1 collapse into 7 such patterns; on real graphs the collapse reaches six orders of magnitude. The peel runs unchanged on patterns, with every step carrying a multiplicity. Count as a coefficient. The number of s -cliques containing a pattern is a coefficient of one small polynomial derived from the quotient. Both the initial counts and all later decrements reduce to coefficient extraction, so no s -clique is ever listed. Certify instead of recompute. Two transfer theorems link neighboring cells. Along s , a pattern whose value reaches its closed form keeps that form in every later cell. Along r , the starting values of a cell follow from the cell below it. Together they leave a single cell to peel; every other cell of the plane is arithmetic.

The consequence is an index that stores almost nothing. Wherever the closed form is certified, the value is reconstructed at query time from the quotient itself, so certified patterns cost zero bytes. We call the structure NSI, for Nucleus Spectrum Index.

Contributions. Our key contributions are summarized as follows. The first index over the nucleus plane. We propose NSI (Section 7), the first index that answers every cell of a bounded (r, s) plane from one structure. On web-it-2004, the explicit table behind the same queries would occupy 46.2 TB, and it cannot be built at all for $r \geq 4$. NSI occupies 1.66 MB and answers a point query in 74

nanoseconds. A quotient framework with certified transfer. We develop the clique quotient and its peel (Sections 4–6): interchangeability, coefficient counting, and two transfer certificates that reduce the whole plane to one peeled cell. Disabling the certificates alone slows the build by 22.6 $\times$ on nasasrb, with answers unchanged. An applicability test computable in advance. We characterize exactly when the quotient compresses, by a one-number test computable before any index is built (Section 7). The test predicted every graph of our evaluation, and it also predicted the social networks the evaluation excludes. On soc-pokec the quotient compresses by a factor of only 1.001, and the test reports this without building anything. Comprehensive Experimental Evaluation. We evaluate on eight graphs from four domains, all with at least a million edges (Section 8). Against CND run serially on the same machine, our builder wins every cell of the evaluation grid, by up to 742 $\times$ in time and 226 $\times$ in memory. It also completes the deep cells on which CND exceeds every budget.

2 Preliminaries

3 The State-of-the-Art Approach

4 The Clique Quotient

5 Peeling in the Quotient

6 From One Cell to the Plane

7 The Index and Its Query

8 Experiments

9 Related Work

10 Conclusion

References

- [1] Leman Akoglu, Hanghang Tong, and Danai Koutra. 2015. Graph-based anomaly detection and description: a survey. *Data Mining and Knowledge Discovery* 29, 3 (2015), 626–688.
- [2] Vladimir Batagelj and Matjaž Zaveršnik. 2011. Fast algorithms for determining (generalized) core groups in social networks. *Adv. Data Anal. Classif.* 5, 2 (July 2011), 129–145. doi:10.1007/s11634-010-0079-y
- [3] Pierre Dupont, Jérôme Callut, Grégoire Doooms, Jean-Noël Monette, and Yves Deville. 2006. Relevant subgraph extraction from random walks in a graph. <https://api.semanticscholar.org/CorpusID:15222702>
- [4] David Gibson, Jon Kleinberg, and Prabhakar Raghavan. 1998. Inferring Web communities from link topology. In *Proceedings of the Ninth ACM Conference on Hypertext and Hypermedia: Links, Objects, Time and Space—Structure in Hypermedia Systems: Links, Objects, Time and Space—Structure in Hypermedia Systems* (Pittsburgh, Pennsylvania, USA) (HYPERTEXT '98). Association for Computing Machinery, New York, NY, USA, 225–234. doi:10.1145/276627.276652
- [5] M. Girvan and M. E. J. Newman. 2002. Community structure in social and biological networks. *Proceedings of the National Academy of Sciences* 99, 12 (2002), 7821–7826. arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.122653799 doi:10.1073/pnas.122653799
- [6] Meng Jiang, Peng Cui, Alex Beutel, Christos Faloutsos, and Shiqiang Yang. 2016. Catching Synchronized Behaviors in Large Networks: A Graph Mining Approach. 10, 4 (2016). doi:10.1145/2746403
- [7] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (Washington, D.C.) (KDD '03). Association for Computing Machinery, New York, NY, USA, 137–146. doi:10.1145/956750.956769
- [8] Rong-Hua Li, Lu Qin, Jeffrey Xu Yu, and Rui Mao. 2015. Influential community search in large networks. *Proc. VLDB Endow.* 8, 5 (Jan. 2015), 509–520. doi:10.14778/2735479.2735484
- [9] Fragkiskos D. Malliaros and Michalis Vazirgiannis. 2013. Clustering and community detection in directed networks: A survey. *Physics Reports* 533, 4 (2013), 95–142. doi:10.1016/j.physrep.2013.08.002 Clustering and Community Detection

- in Directed Networks: A Survey.
- [10] Francis J. O'Reilly, Andrea Graziadei, Christian Forbrig, Rica Breckenkamp, Kristine Charles, Swantje Lenz, Christoph Elfmann, Lutz Fischer, Jörg Stülke, and Juri Rappsilber. 2023. Protein complexes in cells by AI-assisted structural proteomics. *Molecular Systems Biology* 19, 4 (2023), e11544. arXiv:https://www.embopress.org/doi/pdf/10.15252/msb.202311544 doi:10.15252/msb.202311544
 - [11] Giulio Rossetti and Rémy Cazabet. 2018. Community Discovery in Dynamic Networks: A Survey. *ACM Comput. Surv.* 51, 2, Article 35 (Feb. 2018), 37 pages. doi:10.1145/3172867
 - [12] Ryan A. Rossi, Nesreen K. Ahmed, Rong Zhou, and Hoda Eldardiry. 2018. Interactive Visual Graph Mining and Learning, Vol. 9. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3200764
 - [13] Ahmet Erdem Sariyuce, C. Seshadhri, Ali Pinar, and Umit V. Catalyurek. 2015. Finding the Hierarchy of Dense Subgraphs using Nucleus Decompositions. In *Proceedings of the 24th International Conference on World Wide Web* (Florence, Italy) (WWW '15). International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 927–937. doi:10.1145/2736277.2741640
 - [14] Ahmet Erdem Sariyuce, C. Seshadhri, Ali Pinar, and Umit V. Catalyurek. 2017. Nucleus Decompositions for Identifying Hierarchy of Dense Subgraphs. *ACM Trans. Web* 11, 3, Article 16 (July 2017), 27 pages. doi:10.1145/3057742
 - [15] Stephen B. Seidman. 1983. Network structure and minimum degree. *Social Networks* 5, 3 (1983), 269–287. doi:10.1016/0378-8733(83)90028-X
 - [16] Jia Wang and James Cheng. 2012. Truss decomposition in massive networks. *Proc. VLDB Endow.* 5, 9 (May 2012), 812–823. doi:10.14778/2311906.2311909
 - [17] Wenqian Zhang, Zhengyi Yang, Dong Wen, Yi Ding, Wenjie Zhang, and Xuemin Lin. 2026. Nucleus Decomposition Revisited: An Efficient Counting-Based Approach. *Proc. ACM Manag. Data* 4, 3, Article 215 (May 2026), 26 pages. doi:10.1145/3802092
 - [18] Feng Zhao and Anthony K. H. Tung. 2012. Large scale cohesive subgraphs discovery for social network visual analysis. *Proc. VLDB Endow.* 6, 2 (Dec. 2012), 85–96. doi:10.14778/2535568.2448942